

Research Notes on Gradient descent

Gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

>> Section 1

so the sequence $(\mathbf{x}_n)$ converges to the desired local minimum. Note that the value of the step size η is allowed to change at every iteration. It is possible to guarantee the convergence to a local minimum under certain assumptions on the function f (for example, f convex and ∇f Lipschitz) and particular choices of η . Those include the sequence

>> Section 2

Choosing the step size and descent direction Since using a step size η that is too small would slow convergence, and a η too large would lead to overshoot and divergence, finding a good setting of η is an important practical problem. Philip Wolfe also advocated using "clever choices of the [descent] direction" in practice. While using a direction that deviates from the steepest descent direction may seem counter-intuitive, the idea is that the smaller slope may be compensated for by being sustained over a much longer distance. To reason about this mathematically, consider a direction $\mathbf{p}_n$ and step size η_n and consider the more general update:

>> Section 3

Extensions Gradient descent can be extended to handle constraints by including a projection onto the set of constraints. This method is only feasible when the projection is efficiently computable on a computer. Under suitable assumptions, this method converges. This method is a specific case of the forward-backward algorithm for monotone inclusions (which includes convex programming and variational inequalities). Gradient descent is a special case of mirror descent using the squared Euclidean distance as the given Bregman divergence.

>> Section 4

$J_G(\mathbf{x}) = \begin{bmatrix} 3 \sin(\mathbf{x}_2 \times 3) \times 3 \sin(\mathbf{x}_2 \times 3) \times 2.8 \times 1 - 1250 \times 2 + 20 - \mathbf{x}_2 \exp(-\mathbf{x}_1 \times 2) - \mathbf{x}_1 \exp(-\mathbf{x}_1 \times 2) \\ 20 \end{bmatrix}$.
 $3 \sin(\mathbf{x}_2 \times 3) \times 3 \sin(\mathbf{x}_2 \times 3) \times 2.8 \times 1 - 1250 \times 2 + 20 - \mathbf{x}_2 \exp(-\mathbf{x}_1 \times 2) - \mathbf{x}_1 \exp(-\mathbf{x}_1 \times 2)$

>> Section 5

Geometric behavior and residual orthogonality In steepest descent applied to solving $A\mathbf{x} = \mathbf{b}$, where A is symmetric positive-definite, the residual vectors $\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k$ are orthogonal across iterations:

>> Section 6

The basic intuition behind gradient descent can be illustrated by a hypothetical scenario. People are stuck in the mountains and are trying to get down (i.e., trying to find the global minimum). There is heavy fog such that visibility is extremely low. Therefore, the path down the mountain is not visible, so they must use local information to find the minimum. They can use the method of gradient descent, which involves looking at the steepness of the hill at their current position, then proceeding in the direction with the steepest descent (i.e., downhill). If they were trying to find the top of the mountain (i.e., the maximum), then they would proceed in the direction of steepest ascent (i.e., uphill). Using this method, they would eventually find their way down the mountain or possibly get stuck in some hole (i.e., local minimum or saddle point), like a mountain lake. However, assume also that the steepness of the hill is not immediately obvious with simple observation, but rather it requires a sophisticated instrument to measure, which the people happen to have at that moment. It takes quite some time to measure the steepness of the hill with the instrument. Thus, they should minimize their use of the instrument if they want to get down the mountain before sunset. The difficulty then is choosing the frequency at which they should measure the steepness of the hill so as not to go off track. In this analogy, the people represent the algorithm, and the path taken down the mountain represents the sequence of parameter settings that the algorithm will explore. The steepness of the hill represents the slope of the function at that point. The instrument used to measure steepness is differentiation. The direction they choose to travel in aligns with the gradient of the function at that point. The amount of time they travel before taking another measurement is the step size.

>> Section 7

Fast gradient methods Yurii Nesterov has proposed a simple modification that enables faster convergence for convex problems and has been since further generalized. For unconstrained smooth problems, the method is called the fast gradient method (FGM) or the accelerated gradient method (AGM). Specifically, if the differentiable function f is convex and ∇f is Lipschitz, and it is not assumed that f is strongly convex, then the error in the objective value generated at each step k by the gradient

Research Notes on Gradient descent

Gradient descent

descent method will be bounded by $O(k^{-1})$. Using the Nesterov acceleration technique, the error decreases at $O(k^{-2})$. It is known that the rate $O(k^{-2})$ for the decrease of the cost function is optimal for first-order optimization methods. Nevertheless, there is the opportunity to improve the algorithm by reducing the constant factor. The optimized gradient method (OGM) reduces that constant by a factor of two and is an optimal first-order method for large-scale problems. For constrained or non-smooth problems, Nesterov's FGM is called the fast proximal gradient method (FPGM), an acceleration of the proximal gradient method.

>> Section 8

This inequality implies that the amount by which we can be sure the function f is decreased depends on a trade off between the two terms in square brackets. The first term in square brackets measures the angle between the descent direction and the negative gradient. The second term measures how quickly the gradient changes along the descent direction. In principle inequality (1) could be optimized over $\mathbf{p}_n$ and η_n to choose an optimal step size and direction. The problem is that evaluating the second term in square brackets requires evaluating $\nabla f(\mathbf{a}_n - \eta_n \mathbf{p}_n)$, and extra gradient evaluations are generally expensive and undesirable. Some ways around this problem are: